

AWS Well-Architected Framework Review: ROCA New Member Onboarding Flow

1 Executive Summary

The AWS Well-Architected review of ROCA's 7-step New Member Onboarding flow reveals a fundamentally sound serverless architecture—Lambda, API Gateway, RDS Multi-AZ, S3/CloudFront, and SQS—but identifies **15 architectural gaps across all six pillars**, with Security and Reliability carrying the most critical risks that demand immediate remediation.

The most urgent finding is that the onboarding state machine (INVITED → REGISTERED → PROFILE_PENDING → APPROVED → INTERESTS_SET → ACTIVE) is orchestrated entirely in application code rather than through AWS Step Functions, leaving the 72-hour admin approval timeout vulnerable to silent failure during deployments or cold starts [1, 2]. Migrating to Step Functions Standard Workflows would simultaneously resolve gaps in three pillars: Operational Excellence (built-in execution history and observability), Reliability (durable Wait states and Retry/Catch error handling), and Cost Optimization (zero compute charges during approval waits) [3, 4, 5].

Second, the authentication layer has critical vulnerabilities: 7-day JWT refresh tokens lack rotation and reuse detection, OAuth flows do not enforce PKCE, and registration endpoints have no WAF protection against automated abuse. Adopting Amazon Cognito with its native refresh token rotation [6], deploying WAF ACFP managed rules on /auth endpoints [7], and enabling GuardDuty malware scanning on the S3 profile photo bucket [8] are essential to securing member PII—particularly the family and address data stored in unencrypted JSONB columns [9, 10].

Third, the absence of dead-letter queues on SQS consumers means approval reminder and notification messages can be silently lost [11], while the lack of an EventBridge event bus forces synchronous cross-flow integration with Discovery, Volunteering, and Notifications services, increasing coupling and wasting compute [12]. A phased 90-day roadmap—Step Functions and Cognito migration within 30 days, WAF/GuardDuty/EventBridge within 60, and HTTP API migration with DynamoDB on-demand state storage within 90—would bring the onboarding flow to a well-architected posture while reducing API costs by approximately 70% [13] and workflow costs by up to 96% for high-volume sub-tasks [5].

2 Introduction and Architecture Overview

The ROCA (Religious Organization Calendar Application) New Member Onboarding flow implements a sophisticated 7-step progressive journey that guides new members from initial contact to full community integration. The flow encompasses a Welcome Screen and Church Selection, Basic Profile creation (name, email, photo), a Pending Approval gate with admin review, Profile Completion (family and address data), Interests and Serving selection (ministry preferences), Notification and Calendar Sync configuration, and a Welcome Tour [14]. Underpinning this journey is a 10-state member state machine that transitions through INVITED → REGISTERED → PROFILE_PENDING → APPROVED → INTERESTS_SET → ACTIVE, with branch and terminal states including EXPIRED, ABANDONED, REJECTED, APPEALED, RE_REVIEWED, and DEACTIVATED [14].

Chart Explanation: This diagram maps the 10-state onboarding lifecycle to recommended AWS services at each transition point. The main flow (blue) progresses through six primary states, while error and branch states (red/orange) handle failure scenarios. Purple labels indicate the AWS service best suited to orchestrate each phase—from Cognito for authentication through EventBridge for cross-flow event propagation.

The current infrastructure runs on a serverless foundation: Node.js with Express on AWS Lambda behind API Gateway, PostgreSQL 15 on RDS with Multi-AZ deployment, Redis 7 on ElastiCache for caching, S3 with CloudFront CDN for file storage (profile photos), and SQS for asynchronous notification processing [9]. Authentication relies on JWT Bearer tokens with 15-minute access tokens and 7-day refresh tokens, supporting OAuth 2.0 via Google and Apple Sign-in, with role-based access control across five roles (visitor, member, leader, pastor, admin) and organization-scoped data isolation for multi-tenancy [15]. The API surface spans approximately 100 endpoints across 15 groups, with /auth (register, login, refresh, logout, OAuth, password-reset) and /organizations (CRUD, settings, denomination config) being the most relevant to onboarding [9].

Cross-flow integration is a defining characteristic of this architecture. When onboarding completes, the interests set during Step 5 influence the Discovery flow (Flow 3), serving preferences propagate to the Volunteering flow (Flow 4), and notification configuration feeds the Notifications flow (Flow 7). Member approval triggers state changes across Onboarding, Discovery, and Volunteering simultaneously [14]. Notification delivery spans four channels—Push via FCM/APNs, Email via AWS SES, SMS via AWS SNS, and In-app via WebSocket—with smart delivery features including quiet hours (9 PM–8 AM), frequency caps (maximum 5 push notifications per day), and ML-predicted optimal send times [15].

ROCA Member Onboarding State Machine with AWS Service Mapping

Main Flow (blue) · Error/Branch States (red/orange) · AWS Services (purple)

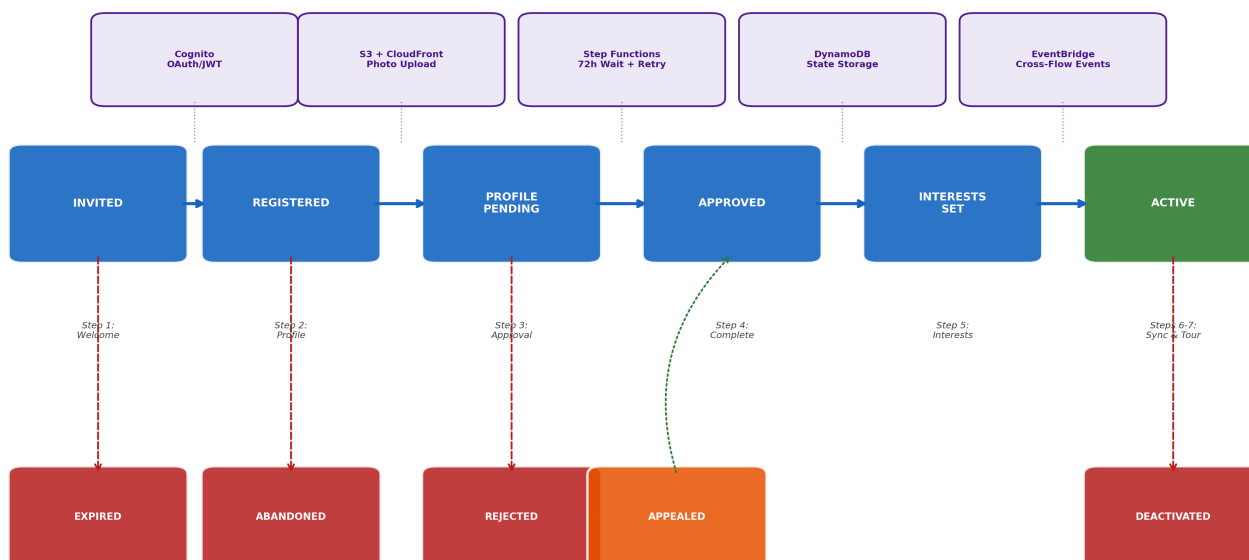


Figure 1: ROCA Member Onboarding State Machine

This review evaluates the onboarding architecture across all six AWS Well-Architected Framework pillars, identifying strengths, gaps, and risks, and providing prioritized recommendations with specific AWS service guidance.

3 Operational Excellence

Current Strengths. The ROCA onboarding flow benefits from a well-defined state machine with explicit transitions and error states, which provides a strong conceptual foundation for operational management [14]. The use of Lambda and API Gateway inherently provides CloudWatch Logs for each invocation, and the SQS-based async processing creates natural boundaries for monitoring queue depth and processing latency.

Identified Gaps and Risks. The most significant operational gap is the absence of explicit workflow orchestration via AWS Step Functions. The current state machine logic is embedded in application code rather than externalized into a managed orchestration service, which means state transitions lack built-in observability, audit trails, and automated error recovery [1]. Without distributed tracing (AWS X-Ray), it is difficult to trace a single onboarding journey across the multiple Lambda functions, SQS queues, and database calls that compose the flow. The 72-hour admin approval timeout with auto-reminders [14] is particularly vulnerable — if implemented as application-level timers rather than durable workflow waits, a Lambda cold start failure or deployment could silently drop pending approvals.

There are no documented automated runbooks for common failure states such as EXPIRED or ABANDONED transitions. When an invite code expires or a member abandons registration, the operational team lacks automated remediation procedures, requiring manual investigation and intervention [1].

Recommendations. First, migrate the onboarding state machine to AWS Step Functions Standard Workflows, which support executions up to one year and provide built-in execution history for 90 days [4]. The 72-hour admin approval should use a Step Functions Wait state, which suspends execution without compute charges and automatically resumes for timeout handling [2]. Second, enable AWS X-Ray distributed tracing across all Lambda functions, API Gateway, and SQS consumers to provide end-to-end visibility into each onboarding journey. Third, create CloudWatch dashboards with key metrics: onboarding completion rate by step, average time-to-approval, abandonment rate by state, and error rates per transition. Fourth, implement AWS Systems Manager runbooks for automated remediation of common failure states — for example, automatically sending re-invitation emails when members enter the EXPIRED state [1].

4 Security

Current Strengths. The authentication architecture demonstrates several sound practices. The 15-minute access token expiration enforces short-lived credentials validated via the exp claim on every request [16]. OAuth 2.0 integration with Google and Apple Sign-in provides secure social authentication, and the five-role RBAC model (visitor, member, leader, pastor, admin) with organization-scoped data isolation establishes a solid multi-tenancy foundation [15]. The organizationCode field in registration provides invite code validation as a gating mechanism [9].

Identified Gaps and Risks. Several critical security gaps require immediate attention. The 7-day refresh tokens currently lack rotation with automatic reuse detection. Amazon Cognito added native refresh token rotation support in April 2025 [6], and this capability should be adopted immediately. Without rotation, a stolen refresh token grants persistent access — as demonstrated in the 2025 Salesloft breach, where compromised refresh tokens enabled unauthorized access to over 700 organizations [17]. The Auth0 best practice of revoking old tokens upon each valid exchange and issuing new ones should be the baseline [18].

For OAuth 2.0 social sign-in, PKCE (Proof Key for Code Exchange) must be enforced for all public clients to prevent authorization code interception, and the state parameter must use cryptographically random values to prevent CSRF attacks [19]. JWTs should be stored in HTTP-only cookies rather than localStorage to prevent XSS exposure [20]. The IETF JWT Best Current Practices (updating RFC 8725) mandate explicit algorithm validation, audience restriction, and issuer checks to prevent algorithm confusion and token substitution attacks [21].

Profile photo uploads to S3 currently lack malware scanning. AWS GuardDuty Malware Protection for S3, which expanded file size and archive scanning limits in September 2025, should be enabled to scan all uploaded files before they are served via CloudFront [8]. For PII protection, the family and address data stored in the users table's profile JSONB column [9] requires encryption with dedicated KMS keys. AWS Prescriptive Guidance recommends at least one KMS key per data classification level, with encryption context for tenant-scoped access control [10].

The registration and invite-code validation endpoints are exposed to automated abuse. AWS WAF's ACFP (Account Creation Fraud Prevention) managed rule group is specifically designed to inspect requests to account registration endpoints and should be deployed [7]. For multi-tenant isolation, the current pool model (shared database with organization-scoped queries) places the burden of isolation entirely on the application layer [20]. AWS documentation recommends either per-tenant user pools for maximum isolation or Cognito groups for RBAC within a shared pool, with 1:1 IdP-to-app-client mapping to prevent unauthorized cross-tenant access [22, 23].

Recommendations. Migrate to Amazon Cognito User Pools with refresh token rotation enabled, PKCE enforcement for all OAuth flows, and group-based RBAC [24, 6, 25]. Deploy WAF ACFP rules on registration endpoints [7]. Enable GuardDuty S3 malware scanning for the profile photo bucket [8]. Implement KMS encryption with separate keys for PII-classified data (family, address, phone) versus general profile data [10].

5 Reliability

Current Strengths. The PostgreSQL 15 RDS Multi-AZ deployment provides database-level high availability with automatic failover [9]. The use of SQS for asynchronous notification processing decouples the onboarding flow from downstream notification delivery, preventing notification failures from blocking state transitions. The explicit error states in the state machine (EXPIRED, ABANDONED, REJECTED, APPEALED) demonstrate thoughtful failure mode design [14].

Identified Gaps and Risks. The most critical reliability risk is the absence of dead-letter queues (DLQs) on SQS consumers. The AWS Well-Architected Serverless Applications Lens explicitly warns that when asynchronous calls fail without DLQs, data loss occurs silently [11]. If a notification Lambda fails to process an approval reminder message, that message is lost, and the admin never receives the 72-hour timeout reminder — potentially leaving members stranded in PROFILE_PENDING indefinitely.

The 72-hour admin approval timeout, currently implemented without Step Functions orchestration, lacks fault-tolerant retry and catch mechanisms. AWS best practices prescribe exponential backoff with jitter for retries, limiting the maximum number of retry attempts to prevent cascading failures [3]. Without intermediate state checkpointing, any failure in the approval pipeline requires a full re-run rather than targeted retry of the failed step [26].

The database schema stores onboarding_status as an ENUM on the users table [9], but there is no documented mechanism for ensuring consistency between the application-level state machine and the database state during concurrent operations or partial failures. A Lambda timeout during a state transition could leave the database in an inconsistent state.

Recommendations. Add DLQs to every SQS queue in the onboarding pipeline, with CloudWatch alarms on DLQ message count to trigger immediate investigation [11]. Migrate the approval workflow to Step Functions with Retry (exponential backoff, max attempts) and Catch blocks that route failures to specific error-handling

states [3]. Implement idempotent state transitions using conditional database updates (e.g., `UPDATE users SET onboarding_status = 'APPROVED' WHERE id = ? AND onboarding_status = 'PROFILE_PENDING'`) to prevent race conditions. Consider DynamoDB for state storage alongside RDS, using its conditional writes for atomic state transitions [27].

6 Performance Efficiency

Current Strengths. The existing architecture leverages several performance-oriented services: CloudFront CDN for profile photo delivery (with free data transfer from S3 to CloudFront) [28], ElastiCache Redis for caching, and Lambda’s auto-scaling for handling variable onboarding traffic [9]. The separation of file storage (S3) from application data (RDS) follows AWS best practices for workload-appropriate storage selection.

Identified Gaps and Risks. The current REST API Gateway configuration is suboptimal for the onboarding use case. HTTP APIs provide native Cognito JWT authorizer integration at approximately \$1.00 per million requests compared to approximately \$3.50 per million for REST APIs — a roughly 70% cost reduction that also comes with lower latency [13]. For profile photo uploads, presigned S3 URLs generated by Lambda should be used to bypass API Gateway’s payload size limits and reduce Lambda execution time [28].

The onboarding flow does not currently leverage the nested workflow pattern recommended by AWS. Standard Workflows should handle the long-running approval process, while Express Workflows should be nested for high-throughput, idempotent sub-tasks such as sending notification batches or validating invite codes [29]. Express Workflows can yield up to 96% cost savings compared to Standard for short-duration, high-volume operations [5].

Calendar sync integration requires a provider-adapter pattern to handle the divergent sync mechanisms across providers. Google Calendar API uses push notifications via watch channels with incremental syncToken-based sync [30]. Microsoft Graph uses delta queries with cursor-based change tracking for Outlook [31]. Apple Calendar relies on CalDAV with ctag/etag polling since Google dropped CalDAV support [32]. The recommended serverless architecture uses API Gateway as the webhook receiver, Lambda for sync logic, DynamoDB for per-user sync tokens and delta links, Secrets Manager for OAuth tokens, and EventBridge Scheduler for CalDAV polling intervals and watch channel renewal [33].

Recommendations. Migrate from REST API to HTTP API for all onboarding endpoints [13]. Implement presigned S3 URLs for profile photo uploads with Lambda-based image processing (Sharp library) for thumbnail generation [28]. Adopt the nested workflow pattern with Standard Workflows for approval orchestration and Express Workflows for notification dispatch and validation sub-tasks [29]. Implement the provider-adapter calendar sync architecture with DynamoDB state storage and EventBridge Scheduler for polling [33]. Consider Lambda durable functions (announced December 2025) for multi-step calendar sync workflows that can checkpoint progress without compute charges [34].

7 Cost Optimization

Current Strengths. The serverless foundation (Lambda, API Gateway, SQS) provides inherent pay-per-use pricing that aligns well with the variable and often bursty nature of onboarding traffic — churches may see spikes around membership drives or seasonal events. S3 and CloudFront provide cost-effective file storage and delivery [9].

Identified Gaps and Risks. The current architecture does not fully exploit available free tiers and cost optimization strategies. The following table summarizes key AWS free tier allowances relevant to the onboarding flow:

AWS Service	Free Tier Allowance	Onboarding Relevance
Step Functions	4,000 state transitions/month [5]	State machine orchestration
Amazon SES	3,000 messages/month (first 12 months) [35]	Welcome emails, approval reminders
DynamoDB	25 GB storage, on-demand reads/writes [27]	Member state storage
CloudFront	1 TB transfer/month [35]	Profile photo delivery
EventBridge	AWS service events free; \$1.00/million custom events [5]	Cross-flow event routing

Table: AWS Free Tier Allowances for Onboarding Services

Step Functions Standard Workflows can suspend execution for up to one year for human approvals without incurring compute charges [2], making them ideal for the 72-hour admin approval wait — the execution simply pauses

and resumes at near-zero cost. For high-volume sub-workflows (notification dispatch, batch validations), Express Workflows at approximately \$0.012 per 1,000 executions versus Standard's \$0.025 per 1,000 state transitions can deliver up to 96% savings [5].

DynamoDB on-demand capacity mode is recommended for onboarding state storage because it requires no capacity planning, scales instantly, and charges only for actual reads and writes — with zero cost when no traffic is flowing [27, 36]. This is particularly advantageous for church applications where onboarding traffic is highly variable.

Recommendations. Adopt Step Functions Standard for the approval workflow (leveraging free-tier transitions and zero-cost wait states) with Express sub-workflows for high-volume tasks [2, 5]. Use DynamoDB on-demand for member state storage [27]. Migrate to HTTP APIs for approximately 70% API cost reduction [13]. Leverage SES free tier for transactional onboarding emails [35]. Write narrow EventBridge rule patterns to avoid unnecessary Lambda invocations that add cost [5].

8 Sustainability

Current Strengths. The serverless architecture inherently supports sustainability by allocating compute resources only during active request processing, avoiding the idle resource consumption of always-on servers [37]. The use of managed services (RDS, ElastiCache, S3) shares underlying infrastructure across AWS customers, improving per-transaction energy efficiency [38].

Identified Gaps and Risks. The current architecture does not fully embrace event-driven patterns. The SQS-based notification processing is a step toward asynchronous design, but the absence of EventBridge means that cross-flow integration (Onboarding → Discovery, Volunteering, Notifications) likely relies on synchronous API calls or polling, which wastes compute cycles [14]. Components that may have become idle or underutilized over time — such as ElastiCache nodes during low-traffic periods — should be reviewed and right-sized [39].

The AWS Sustainability Pillar prescribes maximizing utilization by right-sizing workloads and establishing sustainability goals to reduce compute and storage resources per transaction [38]. For workflow orchestration specifically, AWS recommends using Step Functions with event-based serverless pipelines instead of maintaining compute infrastructure around the clock, and optimizing software for asynchronous and scheduled jobs [39].

Recommendations. Adopt Amazon EventBridge as the central event bus for cross-flow integration, publishing domain events such as `member.registered`, `member.approved`, and `onboarding.completed` to a custom bus with rule-based routing to downstream services [12]. EventBridge Pipes can create point-to-point integrations with optional transform, filter, and enrich steps, reducing the need for intermediate Lambda functions [40]. Establish data contracts for event producers and consumers to validate events and auto-generate bindings [41]. Use EventBridge Scheduler for recurring tasks (CalDAV polling, watch channel renewal) instead of always-on cron processes [33]. Review and consolidate underutilized components, and use SQS buffering to smooth burst traffic and prevent retry storms that waste compute [5].

9 Prioritized Recommendations and Implementation Roadmap

This section consolidates all findings into a prioritized implementation roadmap. The following chart visualizes the distribution and severity of identified gaps across all six pillars.

Chart Explanation: This chart shows the number of identified architectural gaps and risks for each Well-Architected pillar, categorized by severity. Security has the highest concentration of critical gaps (3), driven by missing refresh token rotation, absent malware scanning, and lack of WAF protection on registration endpoints. Reliability follows with 2 critical gaps related to missing DLQs and fault-tolerant timeout orchestration.

9.1 Priority 1 — Critical (Implement Within 30 Days)

The highest-priority items address security vulnerabilities and data loss risks. Migrating the onboarding state machine to AWS Step Functions Standard Workflows with `Wait`, `Retry`, and `Catch` states resolves both the reliability gap in approval timeout handling and the operational excellence gap in workflow observability [3, 4]. Simultaneously, implementing Amazon Cognito with refresh token rotation (available since April 2025) and PKCE enforcement for all OAuth flows closes the most critical authentication vulnerabilities [6, 19]. Adding DLQs to all SQS queues prevents silent message loss in the notification pipeline [11].

9.2 Priority 2 — High (Implement Within 60 Days)

The second tier focuses on threat protection and event-driven architecture. Deploying AWS WAF with ACFP managed rules on registration and invite-code endpoints protects against automated abuse [7]. Enabling GuardDuty

Architectural Gaps by AWS Well-Architected Pillar

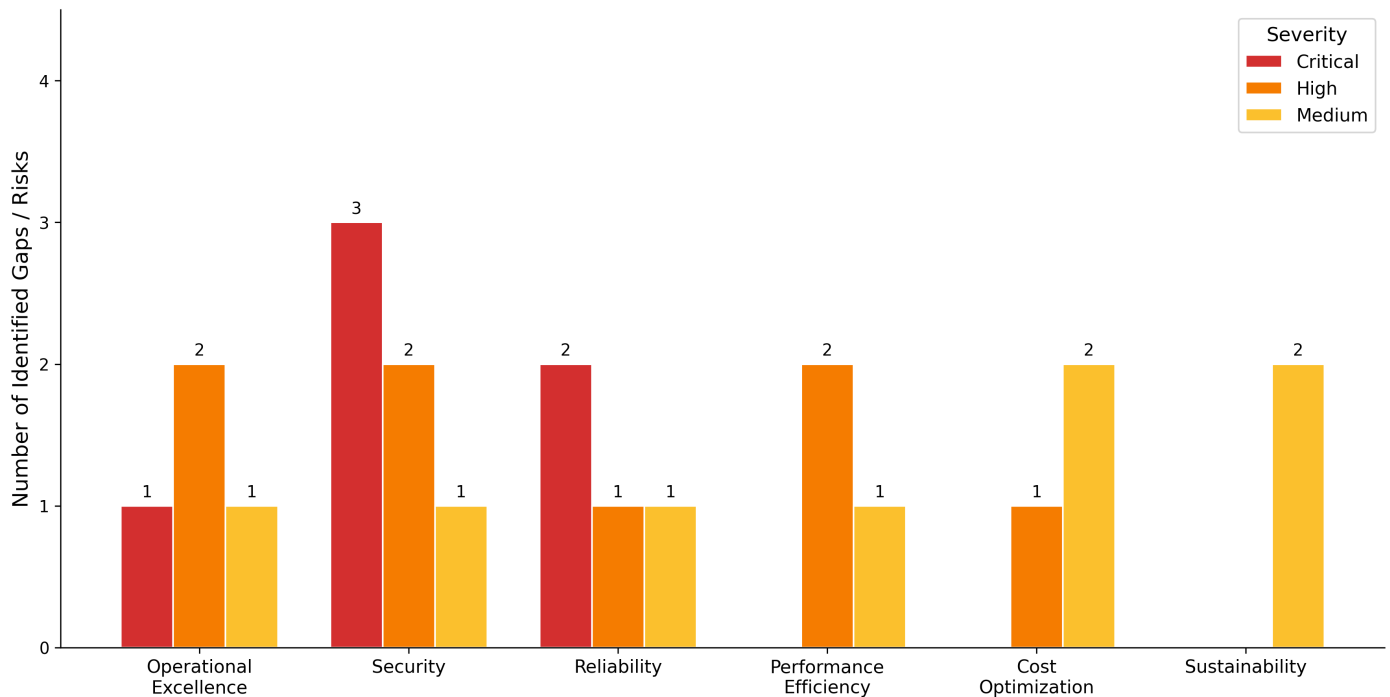


Figure 2: Architectural Gaps by Pillar

Malware Protection for S3 on the profile photo bucket prevents malicious file uploads from reaching CloudFront [8]. Implementing KMS encryption with per-classification keys for PII data (family, address, phone) in the users table strengthens data protection [10]. Deploying an Amazon EventBridge custom event bus for cross-flow integration decouples onboarding from downstream services and enables sustainable event-driven patterns [12].

9.3 Priority 3 — Medium (Implement Within 90 Days)

The third tier addresses performance and cost optimization. Migrating from REST API to HTTP API provides approximately 70% cost reduction with lower latency [13]. Adopting Express Workflows for high-volume sub-tasks (notification dispatch, validation) within the Standard Workflow yields up to 96% savings on those operations [5]. Implementing DynamoDB on-demand for member state storage eliminates capacity planning overhead [27]. Building the provider-adapter calendar sync architecture with DynamoDB state storage and EventBridge Scheduler completes the notification and calendar integration [30, 31, 32, 33]. Finally, conducting a sustainability review to right-size ElastiCache nodes and consolidate underutilized components reduces the per-transaction resource footprint [38, 39].

The following table summarizes the complete recommendation set with AWS service mappings:

Priority	Recommendation	AWS Services	Pillar(s) Addressed
P1	Step Functions state machine migration	Step Functions Standard + Express	Operational Excellence, Reliability
P1	Cognito with refresh token rotation and PKCE	Cognito, Secrets Manager	Security
P1	Dead-letter queues on all SQS consumers	SQS, CloudWatch Alarms	Reliability
P2	WAF ACFP rules on registration endpoints	WAF	Security
P2	GuardDuty malware scanning for S3 uploads	GuardDuty, S3	Security
P2	KMS per-classification encryption for PII	KMS	Security
P2	EventBridge custom bus for cross-flow events	EventBridge, EventBridge Pipes	Sustainability, Operational Excellence

Priority	Recommendation	AWS Services	Pillar(s) Addressed
P3	HTTP API migration (from REST API)	API Gateway HTTP API	Performance Efficiency, Cost Optimization
P3	Express sub-workflows for high-volume tasks	Step Functions Express	Cost Optimization, Performance Efficiency
P3	DynamoDB on-demand for state storage	DynamoDB	Cost Optimization, Reliability
P3	Calendar sync provider-adapter architecture	DynamoDB, EventBridge Scheduler, Secrets Manager	Performance Efficiency
P3	Sustainability review and component right-sizing	CloudWatch, Compute Optimizer	Sustainability

Table: Prioritized Recommendations with AWS Service Mappings

By implementing these recommendations in priority order, the ROCA onboarding flow will achieve a well-architected posture across all six pillars — securing member data, ensuring reliable state transitions, optimizing costs for variable church workloads, and establishing a sustainable event-driven foundation for cross-flow integration with Discovery, Volunteering, and Notification services.

10 References

1. <https://docs.aws.amazon.com/wellarchitected/latest/framework/definitions.html>
2. <https://aws.amazon.com/blogs/compute/best-practices-for-lambda-durable-functions-using-a-fraud-detection-example/>
3. https://docs.aws.amazon.com/wellarchitected/2023-04-10/framework/rel_mitigate_interaction_failure_limit_retries.html
4. <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html>
5. <https://blog.easecloud.io/cost-optimization/step-functions-and-eventbridge-cost-optimization/>
6. <https://aws.amazon.com/about-aws/whats-new/2025/04/amazon-cognito-refresh-token-rotation/>
7. <https://docs.aws.amazon.com/waf/latest/developerguide/waf-managed-protections-best-practices.html>
8. <https://aws.amazon.com/about-aws/whats-new/2025/09/malware-protection-s3-file-size-archive-scanning-limits/>
9. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=8ed3e5b4f-4464-bf0e-ad8ba2c4e4b5>
10. <https://docs.aws.amazon.com/prescriptive-guidance/latest/aws-kms-best-practices/data-protection-encryption.html>
11. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/failure-management.html>
12. <https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-integrating-microservices/event-bridge.html>
13. <https://aws.amazon.com/step-functions/pricing?c=sl&p=ft&z=3>
14. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=9bf52b11-c4b6-417e-a077-cb09f9214287>
15. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=7b9deabe-27e7-4d21-9daf-66c0a3fb90ff>
16. <https://duendesoftware.com/learn/best-practices-managing-token-expiration-refresh-revocation-in-web-apis>
17. <https://www.obsidiansecurity.com/blog/what-are-refresh-tokens-secure-usage>
18. <https://auth0.com/blog/refresh-token-security-detecting-hijacking-and-misuse-with-auth0/>
19. <https://oauth.net/2/pkce/>
20. Source: wikipedia.org

21. <https://www.ietf.org/archive/id/draft-sheffer-oauth-rfc8725bis-01.html>
22. https://docs.aws.amazon.com/cognito/latest/developerguide/bp_user-pool-based-multi-tenancy.html
23. <https://docs.aws.amazon.com/cognito/latest/developerguide/multi-tenancy-security-recommendations.html>
24. <https://docs.aws.amazon.com/cognito/latest/developerguide/>
25. <https://docs.aws.amazon.com/cognito/latest/developerguide/group-based-multi-tenancy.html>
26. <https://docs.aws.amazon.com/wellarchitected/latest/life-sciences-lens/lsrel06-bp01.html>
27. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/on-demand-capacity-mode.html>
28. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/flat-rate-pricing-plan.html>
29. <https://docs.aws.amazon.com/step-functions/latest/dg/sfn-best-practices.html>
30. <https://developers.google.com/calendar/api/guides/sync>
31. <https://learn.microsoft.com/en-us/graph/delta-query-events>
32. <https://cli.nylas.com/guides/sync-calendars-across-providers>
33. <https://docs.aws.amazon.com/lambda/latest/dg/concepts-event-driven-architectures.html>
34. <https://aws.amazon.com/about-aws/whats-new/2025/12/lambda-durable-multi-step-applications-ai-workflows/>
35. [https://aws.amazon.com/ses/pricing/?~:text=The%20starting%20price%20for%20dedicated_threshold%20\(see%20](https://aws.amazon.com/ses/pricing/?~:text=The%20starting%20price%20for%20dedicated_threshold%20(see%20)
36. https://aws.amazon.com/dynamodb/pricing/on-demand/?nc1=h_ls
37. https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sus_sus_software_a2.html
38. <https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/design-principles-for-sustainability-in-the-cloud.html>
39. <https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/software-and-architecture.html>
40. <https://aws.amazon.com/eventbridge/pipes/>
41. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/event-driven-architectures.html>